

Change request log #1

1. Concept Location

Step #	Description	Rationale
1	We ran the system	
2	We interacted with the system: after logging in we entered the schedule screen. We opened some text files and looked at them.	To get familiar with some of the features of the system and identify the screens or graphical elements we had to change.
3	We searched for "Status Bar" using the regular expression feature of the IDE's search tool.	Because we need to make the change in the status bar.
4	We then clicked on "StatusBar.java"	We selected this file because there was only one ".java" file named "StatusBar.java", so we clicked on it.
5	We inspected the "StatusBar"	When we inspected the class, we found that the functionality of the class included Sorting Recent files, iterating over recent files and creating a textfield to filter recent files based on user input
6	We marked the "StatusBar.java" class as located	When we looked through the methods of the class, we found 2 methods which had the following functionalities: 1. updateCaretStatus() method updates the status bar with information regarding the current caret position, line number, caret offset, and buffer length. It checks if the buffer is loaded and ensures the caret is within the current buffer; if not, it sets the status bar text to empty. The method calculates the caret's position within the current line and its virtual position considering tab width. It formats the display by appending the caret line number, caret position, and virtual position based on view preferences. Additionally, it appends the caret offset and buffer length to the status bar string, depending on user preferences. 2. propertiesChanged(): This method is called when properties related to the status bar or its components are changed. It updates the appearance and content of the status bar based on the updated properties.

Time spent (in minutes): 30 minutes.

Classes and methods inspected:

- build/classes/core/org/gjt/sp/jedit/gui/StatusBar.class
 - void updateCaretStatus()
 - void propertiesChanged()

2. Impact Analysis

Step #	Description	Rationale
1	Identified potentially impacted classes: StatusBar , Buffer , JEditTextArea .	Visited StatusBar as it is directly responsible for status bar functionality. Buffer and JEditTextArea are key components dealing with text editing and document representation.
2	Reviewed the StatusBar class to understand its current functionality and potential impact.	Understanding existing behavior helps identify areas where modifications may be required and evaluate potential effects on overall system behavior.
3	Reviewed the updateCaretStatus() method within StatusBar for modification feasibility.	This method handles status bar updates related to caret position, making it a prime candidate for incorporating word offset display.
4	Discarded classes Buffer and JEditTextArea from the estimated impact set.	While these classes are closely related to text editing, they are not directly responsible for status bar functionality, reducing their relevance in this particular change

Time spent (in minutes): 45 minutes.

Classes and methods inspected:

- build/classes/core/org/gjt/sp/jedit/gui/StatusBar.class
 - void updateCaretStatus()
 - void propertiesChanged()
- build/classes/core/org/gjt/sp/jedit/Buffer.class
- build/classes/core/org/gjt/sp/jedit/JEditTextArea

3. Actualization

Step #	Description	Rationale
1	Reviewed the existing StatusBar class and its methods, focusing on updateCaretStatus() .	The updateCaretStatus() method is responsible for updating the status bar with caret-related information, making it a suitable candidate for incorporating word offset.
2	Decided to modify the updateCaretStatus() method to calculate and display word offset.	This method already handles updating caret-related information; thus, extending it to include word offset ensures coherence and maintains a centralized approach to status updating.
3	Attempted to modify the code to incorporate word offset directly within the updateCaretStatus() method itself.	It became apparent that directly integrating word offset calculation within this method would result in code complexity and reduce readability. Therefore, the decision was made to introduce a separate helper method, countWordsInBuffer() , to handle word count calculation, ensuring a cleaner and more maintainable implementation.

4	Added a new helper method <code>countWordsInBuffer(Buffer buffer)</code> to calculate the total word count.	A separate method is necessary to count words in the buffer, ensuring modularity, readability, and facilitating future modifications related to word count calculation.
5	Integrated the <code>countWordsInBuffer()</code> method within <code>updateCaretStatus()</code> to obtain the total word count.	Utilizing this helper method ensures accurate calculation of the total number of words in the file, which is essential for displaying word offset correctly.

Time spent (in minutes): 50 minutes.

Classes and methods inspected:

- build/classes/core/org/gjt/sp/jedit/gui/StatusBar.class
 - void updateCaretStatus()

Classes and methods changed:

- build/classes/core/org/gjt/sp/jedit/gui/StatusBar.class
 - void updateCaretStatus()
- build/classes/core/org/gjt/sp/jedit/gui/StatusBar.class
 - countWordsInBuffer()

4. Validation

Step #	Testing Level	Description	Rationale	Result
1	Unit Testing	Unit test the updateCaretStatus method to ensure accurate calculation of word offsets.	Validate the method's precision in determining the word offset from the file's beginning.	Pass - the word offset is accurately calculated.
2	Unit Testing	Unit test the countWordsInBuffer method.	Verify the method's ability to accurately count words within the buffer.	Pass - the word count is precise for the given buffer.
3	Regression Testing	Regression test - the entire status bar functionality post-change implementation.	Ensure the status bar reflects accurate information on line number, column position, word offset, and total words in the file.	Pass - all status bar features function as expected with the modified functionality.
4	Regression Testing	Regression test performance post-change.	Confirm the continued responsiveness and efficiency of the jEdit application with the updated status bar.	Pass - performance remains unaffected.
5	System Testing	System test the entire jEdit application with the updated status bar.	Evaluate the overall functionality of jEdit, focusing on enhancements brought by the modified status bar.	Pass - the jEdit application seamlessly incorporates the updated status bar.
6	System Testing	System test the status bar across various lengths of text files and edge cases.	Ensure the modified status bar handles diverse file contents and edge cases appropriately.	Pass - the status bar behaves correctly in different scenarios.
7	Integration Testing	Integration test the interaction between the status bar and updated features.	Confirm smooth integration between the status bar and new word offset/word count features.	Pass - seamless integration between the status bar and updated functionality.
8	Integration Testing	Integration test the status bar with existing jEdit features.	Ensure compatibility and absence of conflicts with other jEdit features.	Pass - no conflicts or issues observed with concurrent use of the status bar and existing features.
9	Acceptance Testing	Acceptance test the jEdit application, emphasizing the modified status bar.	Evaluate if the requested change has been successfully implemented.	Pass - change successfully executed.

10	Acceptance Testing	Acceptance test the entire jEdit application with the updated status bar functionality.	Confirm overall improvement in the jEdit experience with the modified status bar.	Pass - enhanced jEdit experience with the updated status bar functionality
----	--------------------	---	---	--

Time spent (in minutes): 60 minutes.

5. Summary of the change request

Phase	Time (minutes)	No. of classes inspected	No. of classes changed	No. of methods inspected	No. of methods changes
Concept location	30	1	1	1	1
Impact Analysis	45	3	1	2	2
Actualization	50	1	1	2	2
Validation	60	1	1	2	2
Total	185	3	1	2	2

6. Conclusion

The concept location for this change was very easy because the file is very easy to locate from the status bar itself. As we can understand clearly from the description of the change, that the issue to be updated would be in Status bar. The modification to the **StatusBar** class in jEdit has significantly improved the user experience by introducing the display of word offset information. By enhancing the **updateCaretStatus()** method, users can now easily track the word offset of the caret from the beginning of the file, along with the total word count in the document. This addition offers valuable insights into the text being edited, allowing users to navigate and manage their documents more efficiently. Additionally, the inclusion of the **countWordsInBuffer()** method ensures accurate word counting within the buffer, further enhancing the reliability of the word offset feature. Overall, these changes represent a significant enhancement to the jEdit text editing environment, empowering users with comprehensive document analysis capabilities.